

ICS 184 / EECS 116 Summer 2006 Assignment #2

Due date and time: *Monday, July 24, 11:59pm*

Introduction

Relational algebra, which we explored in the previous assignment, provides a clean, precise way to specify queries on relational databases, making it a good mathematical tool for conceptualizing database queries. When we move into a practical setting, however, where we need to interact with a computer, the qualities that determine whether something is a "good tool" change. Requirements that come into play include a concise, straightforward syntax, shorthands for commonly-needed operations, and even limiting the tool so that it uses only characters that can be entered easily using a keyboard. Unfortunately, relational algebra falls short by these measures, and though it forms the conceptual basis for much practical work with databases, it does not enjoy much direct popularity in the information technology industry.

SQL is the world's premier language for querying, manipulating, and defining relational databases. In contrast to the limitations of relational algebra in a practical context, SQL has qualities we might look for in an everyday tool, such as a concise, flexible, English-like syntax that can describe complex queries with relatively little "code," ubiquitous support by a wide variety of pre-existing software on virtually any platform, and the ability to connect SQL-based database products into new programs written in many programming languages.

In this assignment, you'll have the opportunity to practice using SQL to write queries, and to execute them using a popular open-source database management system (DBMS) called MySQL. Subsequent assignments will explore other parts of SQL, such as defining constraints on data, specifying views, and implementing triggers and stored procedures.

The database

For this assignment, we'll continue working with a database that underlies the operations of a hypothetical company that manages automobile dealerships throughout the United States. The data requirements of the enterprise are the same as they were in [Assignment #1](#). The database schema is the same, as well; I've reproduced it here for your convenience.

```
car(vin, make, model, year, mileage, asking_price, invoice_price)
dealership(dealership_id, dealership_name, dealership_street_address, dealership_city, dealership_state)
salesperson(salesperson_id, salesperson_name)
customer(social_security_number, customer_name, customer_street_address, customer_city, customer_state)
reports_to(salesperson_id, managing_salesperson_id)
works_at(salesperson_id, dealership_id, month_worked, base_salary_for_month)
inventory(vin, dealership_id)
sale(vin, social_security_number, salesperson_id, dealership_id, sale_price, sale_date)
```

As in the previous assignment, there are implied relationships between data in various relations. For example, the *vin*

attribute of the *sale* relation depends on the *vin* attribute of the *car* relation; in other words, every sale of a car relates to the car that was sold. For this assignment, we'll regard these relationships as implied and will, therefore, manage them ourselves; in a later assignment, we'll learn how to specify them explicitly, so that the DBMS can manage them for us.

The DBMS: MySQL 5.0

Rather than asking you to simply write your SQL queries on paper, I'd like you to gain experience installing and using an actual DBMS. This will allow you to experiment with queries different from the ones I'm asking you to write, as well as test your solutions to the assigned queries before you submit them. It will also force you to confront real-world issues like needing a good set of test data, as well as the problem of DBMS's each supporting a different subset of SQL.

This quarter, we'll be using MySQL 5.0, an open-source DBMS. I've written [a document that explains how to install and use MySQL 5.0](#). It would be a good idea to read that document and be ready to use MySQL before proceeding with the assignment.

The queries

Your primary task in this assignment is to write SQL queries that meet each of the following specifications. Each query is required to execute correctly in MySQL 5.0, using a database created using an SQL script I've provided; the creation and use of SQL scripts is described in the next section of this write-up. Since the queries need to run in MySQL, it is possible that some legal SQL techniques, including some we discussed in lecture, will not be allowed in this assignment. (This is a genuine frustration that often occurs in real-world software development, as well.)

Write MySQL-compatible SQL queries that solve each of the following problems. For each, be sure that you're including all of (and only) the appropriate rows and columns in your result.

1. List the ID's and names of all salespeople in the database.
2. List the Social Security Number, name, street address, and city of each customer in the database who lives in Missouri. (Missouri is denoted in the database by the two-letter postal abbreviation 'MO'.)
3. For each car in the inventory of any dealership, list the VIN, make, model, and year of the car, along with the name, city, and state of the dealership whose inventory contains the car.
4. List the Social Security Numbers and names of all customers who have ever bought cars from the dealership named "Irvine Toyota Sales".
5. List the names of all salespeople who are managed by a salesperson named "Joe Jones".
6. List the ID's and names of all salespeople who do not have a manager.
7. Find the number of dealerships.
8. List the name, street address, city, and state of any customer who has bought more than two cars from all dealerships combined since January 1, 2000.
9. List the five make/model combinations that were sold more than any others in California ('CA') between February 1, 2006 and February 28, 2006, inclusive. For each of these five make/model combinations, also list the number of cars that were sold in that period, and list the results in descending order of the number of cars sold.
10. Find the average of the *total gross sales* amount between October 1, 2005 and December 31, 2005 for all dealerships whose total gross sales amount was at least \$1,000,000. The total gross sales amount for a

dealership within a certain time range is the sum of the sale price of all cars sold by that dealership during that time range.

11. Find the names of all customers who bought cars during 2005 who were also salespeople during 2005. For the purposes of this query, assume that no two people have the same name.
12. List the name, city, and state of the dealership that had the highest average sale price for all cars sold in 2005. Include the average sale price of this dealership in the result. In the event of a tie, include all dealerships with the highest average.
13. Calculate the payroll for the month of March 2006.
 - The payroll consists of the name, salesperson ID, and gross pay for each salesperson who worked that month.
 - The gross pay is calculated as the base salary at each dealership employing the salesperson that month, along with the total commission for the salesperson that month.
 - The total commission for a salesperson in a month is calculated as 7% of the profit made on all cars sold by the salesperson that month.
 - The profit made on a car is the difference between the sale price and the invoice price of the car. (Assume, for simplicity, that cars are never sold for less than the invoice price.)

You may assume that there are no NULL values anywhere in the database.

Building a set of test data

Background

In a production system running in an actual enterprise, a database is most often intended to manage a huge collection of real data. One of the challenges in initially designing and implementing a database is testing it, since the huge collection of real data usually doesn't exist until after the system has gone "live." This necessitates building a set of test data to put into the database during development.

Building a test data set seems relatively simple at first. One idea is just to generate random data. For example, to generate a set of salespeople for our database, we might create 1000 salespeople with consecutive ID's and randomly-generated names, and create 100 dealerships similarly with consecutive ID's and randomly-generated names and addresses.

This kind of random data generation is a good start, but there are open questions. How should we relate the salespeople to the dealerships? Should each dealership have an equal number of salespeople working for it? Should some dealerships be larger than others, and hence have more salespeople? How about cars and sales? Should some kinds of cars sell more often than others, or be sold more often at some dealerships than others? The performance of some queries is sensitive to the distribution of data within a database (e.g., how many customers there are, or how many cars are sold at each dealership), so to get an accurate picture of how efficiently the queries will run in the finished system, it's necessary to make a good estimate about the actual data distribution ahead of time. Handled well, this problem is treated not as only an engineering problem, but also one that depends on understanding business requirements and asking the right non-technical people the right kinds of questions. (Some enterprises hire people specifically for the job of generating good test data; a friend of mine once worked in a large-scale corporate IT environment, in a group that had two employees who were there just to come up with good test data.)

Since we're not intending to do any sort of performance analysis in this assignment, it might seem that we needn't be too concerned about these kinds of issues. However, even for this assignment, generating good test data will be a bit

of a challenge, because it will be important to include data that tests your queries in not only the typical cases, but also the boundary and error cases, as well. For example, in a query that asks how many cars were sold at each dealership in a particular month, what if some dealerships didn't sell any cars at all? Without including such a dealership in your test data set, you won't know whether your query works correctly in that case.

Starting point

To get you started, I've provided an *SQL script* called [automobile.sql](#), which is a text file that consists of a sequence of SQL commands that create an instance of our automobile database and populates it with some data. I didn't construct this set of data carefully, so it will no doubt be missing many useful test cases. I encourage you to update this script as you write your queries, including test data that verifies the behavior of each query that you write. While this is not an explicit requirement in the assignment, and you are not required to submit your test data, it is a good way to be sure that your solutions are correct before you submit them. (I should point out that we have a complete set of test data that I've built to test your queries; we'll be using this set of test data to test your queries after you submit them, so it's in your best interest to thoroughly test your queries before turning them in.)

Executing the provided SQL script

Executing the provided SQL script is relatively straightforward. Place it into a folder, then bring up a command prompt and change into that folder. Then execute the following command:

```
mysql --user=root --password <automobile.sql
```

You'll be asked to type your root password, then the contents of the SQL script **automobile.sql** will be executed as though you'd typed the commands at the **mysql>** prompt. When the commands have finished executing, you'll be brought back to the command prompt, at which time you can start up **mysql** normally, issue the **USE automobile;** command, and interact with your new database.

A word of advice about maintaining your test data

As you build test data, write it into **automobile.sql**, rather than just issuing **INSERT** commands in **mysql** directly. This way, you won't lose your test cases whenever you execute **automobile.sql** (since **automobile.sql** starts by dropping the database and recreating it from scratch).

Using the DATE data type in MySQL

MySQL supports a data type called **DATE**, which allows you to store dates like February 13, 2006 into a database column. The day of the month is optional and can be set to 0, allowing you to store months like February 2006 into a database column, as well. Both of these uses of **DATE** appear in this assignment, because the *sale_date* column of the *sale* table is intended to consist of dates like February 13, 2006, and the *month_worked* column of the *works_at* table is intended to consist of months like February 2006.

There are a few ways to specify dates in MySQL, but I'm asking you to use one particular one, so we can all remain on the same page. Dates can be specified literally as character strings in a '**YYYY-MM-DD**' format. For example, to add a sale taking place on February 13, 2006 into the database, you would use the following syntax:

```
INSERT INTO sale VALUES ('4A57B8HLK25FR134Z', '123456789', 101, 201, 37508.78, '2006-02-13')
```

where '2006-02-13' indicates February 13, 2006. To add a row into the *works_at* table indicating that someone worked

at a dealership during the month of February 2006, you would use **00** for the day of the month, like this:

```
INSERT INTO works_at VALUES (101, 201, '2006-02-00', 3500.00)
```

Dates can be compared using the standard comparison operators such as `=`, `<>` (not equal), and `>=`. One date is considered to be less than another if it's earlier; for example, '2006-02-11' is less than '2006-02-13', since it occurs two days earlier.

A word of caution about SQL

SQL is the most commonly used database definition and manipulation language in the world. Like many programming languages, a series of standards has been developed over the years, each comprising an agreement among the community of SQL users about the syntax and semantics of the language. The latest of these standards is known as SQL:2003. The intent of such standards is to make it possible to move from one language implementation to another, with little or no difference in the behavior of programs; following the standard is a good way, as a programmer, to keep programs reasonably portable between implementations. (Often, though, this isn't enough.)

Most implementations of standardized programming languages (e.g. C++, Java, Scheme) embody the language as specified in its standard — or, at least, a substantial subset of the language, with little deviation from the standard other than a few missing features. (There is only one compiler, for example, that implements the complete C++ standard, though most come very close.) If you write a syntactically and semantically correct program in such a language, then run it through a compiler for that language, you can have a reasonable expectation that the program will compile and run successfully.

Though SQL is standardized, the reality is that no DBMS implements the complete SQL standard, and each introduces its own extensions and quirks. Many are missing a substantial portion of the functionality required by the standard. For this reason, it will be important when working on these assignments to ensure that you're writing SQL statements that are compatible with the DBMS, MySQL 5.0, that we'll be using in this course. If you have prior experience with a different database system, you may find that MySQL supports a different subset of SQL than you're accustomed to, and that it uses slightly different syntax than you've learned in some cases; we will also cover some SQL features in lecture — and you'll read about some in your textbook — that are not supported by MySQL, though I will try to make these distinctions clear along the way.

Be aware that the [MySQL 5.0 Reference Manual](#) is the final arbiter of what is considered to be legal SQL, for the purposes of this assignment, as well as future assignments. There will be no exceptions to this policy.

Deliverables

Write all of your SQL queries into an SQL script named **Assignment2.sql**. *Do not* include anything in your script other than the queries, as well as a comment above each query specifying which of the problems it solves. The script should not include the creation of any test data, or introduce any modifications into the database; the only kind of SQL statement that's permitted in **Assignment2.sql** is SELECT. (We intend to test your queries using our own set of test data, so it's important that your script does not alter our data in any way, or your queries will return the wrong answer.)

Remember that, in SQL scripts, a semicolon is required after each query. This is not something that's part of the SQL language, but is required in SQL scripts as a way of denoting where one query ends and another begins.

Follow [this link](#) for a discussion of how to submit your assignment. Be aware that I'll be holding you to all of the rules specified in that document, including the one that says that you're responsible for submitting the version of the assignment that you want graded. We won't regrade your assignment simply because you submitted the wrong version by accident.

A few modifications, including the requirement of specifying the queries as an SQL script, introduced by Alex Thornton, Spring 2006.
Originally written by Alex Thornton, Winter 2006.